

Principles of programming languages

Indice

Scheme (Racket)	2
Basics	2
Variables and binding	3
Static vs dynamic scoping	3
Homoiconicity	3
Syntactic forms	3
Quoting	4
Eval	4
Begin	4
Define	4
Lists	4
Loops	5
Equality	5
Case and cond	6
Storage model	6
Evaluation strategy	6
Structs	6
Closures	6
Classic higher-order functions	7
Macros	7
Continuations	7
 Playing around: implementing exceptions	 8
Object orientation	8
Closures	9
Prototypes	9
 Haskell	 10
Evaluation	10
Currying	11
Function definition	12
Type system	12
User-defined types	12
Recursive types	12
Fields	13
Synonyms	13
Function composition and \$	13
Infinite computations	13
Error	13
Pattern matching	13
If-then-else	14
let and where	14
Call-by-need and strictness	14
Modules	14

Type classes	15
IO	15
Exceptions	16
Maps and arrays	16
Towards monads	16
Foldable	17
Functor	17
Applicative functors	17
Monads	17
Monadic laws	18
The do notation	18
The list monad	18
The State monad	19
Erlang	19
The basics	19
Function syntax	20
Case and If	21
Lambdas and higher order functions	21
The actor model	21
Concurrent programming	21
Client-server	22
Timeouts	23
Building application using Erlang	23
Rust	24
Memory safety	24
Single ownership	24
Multiple ownership	24
Rust's solution	24

Scheme (Racket)

Based on LISP, “it allows to use the language to explore the design, implementation and semantics of programming languages”. We will use it to build new constructs and an OO language.

Like LISP, it uses s-expressions:

```
(= x (+ y (* 3 x) z)) ; comment
```

Advantages of s-expressions:

1. **It allows arbitrary number of arguments**
2. **The programmer is describing the syntax tree of its program**
 - Very simple compilers/interpreters
 - Metaprogramming is very easy

Basics

Basic types:

1. Booleans: #t, #f
2. Numbers: any precision, floating, complex; expressed with scientific notation or ratio
3. Chars: #\a
4. Symbols: a-symbol?, another-symbol?
5. Vectors: #(1 2 3 4)
6. Strings: "aaa"
7. Pairs and lists: (1 2 #\a)
 - Lists are the basis of the language: everything is a list (LISP), even the program themselves.

Every program is an *expression*: evaluation of expressions produces a value (opposed to statements). The computation in scheme is based on evaluating expressions. The evaluation of `(e1 e2 e3)` is as follows:

1. `e1` (usually a symbol) is evaluated first and identifies an operation `f`
2. The other parameters are evaluated *in any order* and then passed to `f`

Like other functional languages, lambdas are present. Procedures are still values, hence functions are first-class citizens.

```
(lambda (x y)
  (+ (* x x) (* y y)))
```

Variables and binding

`let` binds new variables. It takes a list of 2 element lists containing name value.

```
(let ((x 1)
      (y 2)))
```

Scoping is present and is static (not dynamic like in traditional LISP).

`let` binds variables parallelly, `let*` does so sequentially. If mutual recursion is needed, `letrec` and `letrec*`.

```
(let ((x 1)
      (y 2)
      (let ((x y) ; swaps x and y
            (y x))
        ...x)))
```

; vv this is wrong vv

```
(let ((x 1)
      (y (* 2 x))) ; x is still not defined at this point. We need to use let*
  ...)
```

```
(letrec ((f (lambda () (f))))
```

Static vs dynamic scoping

Consider the following:

```
(let ((a 1))
  (let ((f (lambda ()
              (display a))))
    (let ((a 2))
      (f))))
```

In scheme (static scoping) the result is 1. Static scoping is derived from how logic (like FOL) works. With dynamic scoping the result would be 2.

Homoiconicity

Scheme is **homoiconic**: **there is no distinction between code and data**. We have that code is data, and this is very effective for metaprogramming.

Note: machine code is itself homoiconic: both instruction and data are bytes, the meaning depends on the interpretation

Syntactic forms

Not everything is a procedure or value: e.g. `if` is a syntactic form. Unlike functions, `if` does not evaluate all its arguments. Similarly, `lambda` also is a syntactic form. **We can define new syntax using macros.**

```
(if <condition> <then> <else>)
(when <condition> <then>)
(unless <condition> <else>)
```

Syntactic forms are needed because we cannot create a language using only call-by-value semantics: and if(cond, then, else) can be defined and wrapping each branch in lambdas, but then how can we define lambdas?

Quoting

We can prevent evaluation by “quoting” an expression. quote prevents evaluation, quasiquote and unquote are used for partial evaluation:

1. quasiquote blocks external evaluation
2. unquote forces evaluation

```
(quote <expr>)
('<expr>) ; shorthand for quote
`(1 ,(+ 1 1) 3) ; shorthand for quasiquote-unquote
                ; evaluates to (1 2 3)
```

Eval

eval is function that takes a code and interprets it. It is **effectively the opposite of quote**. *Note: eval is considered dangerous, never use it*

```
(eval '(+ 1 2 3)) ; outputs 6
```

Begin

If we are writing procedural code, we can use the begin construct. It evaluates every operation in order and the return value is the last one of the block

```
(begin
  (op1 ...)
  (op2 ...)
  ...
  (opn ...))
```

Define

define creates top-level bindings. Defining a procedure is done via define

```
(define <name> <what>)

(define x 12)
(define cube (lambda (x) (* x x x))) ; dot use this
(define (cube x) (* x x x)) ; use this for procedures
```

We can also use define in procedures instead of let. For assignment we can use set! (*Note: functions with side-effects are by convention suffixed with !*).

Lists

Lists are the center of the language (LISt Processor). Lists are **stored as linked lists**. **Each node of the chain is pair called a cons node. The first element of the pair is data, and called car (Content of the Address Register), while the second cdr (Content of the Data Register).**

A **pair** is expressed in scheme as (x . y). We can thus write a list as:

```
(1 . (2 . (3 . ())))
() ; the empty list, also called nil
```

car and cdr functions are used as accessors. member checks element presence and returns the cdr of the element.

Defining data using ', **we are creating literal constants**. To create a new pair/lists/arrays we use **constructors**: cons for pairs, list for lists and vector for vectors.

Procedure bodies and parameter lists are plain lists. This is used to implement procedures with a variable number of arguments.

```
(define (f x y) (...)) ; we are defining a function with 2 parameters
(define (f x . y) (...)) ; we are defining a function with a variable number of args
;      ~~~~~ -> we have the first argument bound to x and all the others to y

;      ~~~~~ -> we define a function x with all variables bounded to y
(define (x . y) y)
(x 1 2 3) ; -> '(1 2 3)

apply can be used to apply a procedure to a list of elements.
(apply + '(1 2 3 4)) ; => 10
```

Loops

One non-idiomatic way of looping is using a named let:

```
(let ((x 0))
  (let label ()
    (when (< x 10)
      (display x)
      (newline)
      (set! x (+ x 1))
      (label)))) ; basically a goto label
```

We can make things better by better utilizing the named let:

```
(let label ((x 0))
  (when (< x 10)
    (display x)
    (newline)
    (label (+ x 1)))) ; applies the return value of the last expression to
                      ; the bindings
```

Every scheme implementation is required to be tail-recursive.

```
(define (factorial n)
  (define (fact-rec x acc)
    (if (= x 0)
        acc
        (fact-rec (- x 1) (* x acc))))
  (fact-rec x 1))
```

For each has a peculiar syntax:

```
(for-each
  (lambda (x)
    (display x)
    (newline))
  '(1 2 3 4))
```

Equality

A predicate that returns a boolean by convention ends with ? (e.g. null?). For testing equality between objects we have:

- =: used only for numbers
- eq?: tests if two objects are the same object
- eqv?: like eq? but defined for numbers
- equal?: true iff the unfolding of its arguments into regular trees are equal as ordered trees.

The longer the name, the more demanding the evaluation of the predicate.

Case and cond

case is like a switch, while cond is a generalized if (if-elif chain). The predicate used by case is eqv?

```
(case (car '(c d))
      ((a e i o u) 'vowel)
      ((w y) 'semivowel)
      (else 'consonant))
(cond ((> 3 3) 'greater)
      (< 3 3) 'equal)
      (else 'equal))
```

Storage model

The language is garbage collected. Variables and objects simply refer to locations on the heap. Constants reside in read-only memory, therefore literal constants are immutable. Mutation, if possible, is achieved using ! functions.

In racket, lists are immutable (so no set-car! or set-cdr!), unlike in LISP or other dialects of scheme. There is, however, a mutable pair datatype with mcons, set-mcar! and set-mcdr!.

Evaluation strategy

Scheme uses *call by object sharing*, like java. Objects are allocated on the heap and references to them are passed by values. It is also called *called by value*, because objects are evaluated before the call and such values are copied into the activation record. The copied values are not the object itself, but a reference to it. This means that if the object is mutable, functions can exhibit side effects.

Structs

In scheme we can create structs (like in C). Constructor, predicate for checking type and accessors (for mutable fields also setters) are created automatically on declaration.

```
(struct being (
  name          ; by default fields are immutable
  (age #:mutable) ; if we want to make them mutable we have to specify
))

(define p (being "pippo" 29)) ; `being` is automatically created
(being? p) ; `being?` is automatically created
(being-name p)
(being-age p)
(being-show p) ; like a print
(set-being-age! p 12)
```

Structs can inherit from other structs:

```
(struct may-being being
  ((alive? #:mutable)))
(define (kill! x)
  (if (may-being? x)
      (set-may-being-alive?! x #f)
      (error "not a may-being" x)))
```

The main difference between the struct system and OOP is the difference between procedures and methods: **Procedures are external, so we cannot redefine/override them.**

Closures

A closure is a **function together with a referencing environment for the non-local variables of that function**. They are objects allocated on the heap and can be seen as the basic unit of object orientation (data + behaviour).

Classic higher-order functions

These operations are supported on most languages: map, filter and folds.

Folds are more complicated. We have two of them: foldr and foldl (aka reduce in python). Given $\circ$ a binary operation:

$$\begin{aligned} fold_{\text{left}}(\circ, i, (e_1, \dots, e_n)) &= (e_n \circ (e_{n-1} \circ \dots ((e_1 \circ i))) \\ fold_{\text{right}}(\circ, i, (e_1, \dots, e_n)) &= (e_1 \circ (e_2 \circ \dots (e_n \circ i))) \end{aligned}$$

The **left fold is more efficient, since it is easy to write as tail-recursion. The right fold can be written to be tail recursive, however it can be complicated and the results are not better:** the non-tail recursion uses the stack, while the tail-recursive one uses the heap since it needs to allocate a closure for each invocation.

Macros

Macros are defined through define-syntax and syntax-rules. syntax-rules are pairs of (pattern expansion): pattern is matched by the compiler and expanded into expansion.

```
(define-syntax while
  (syntax-rules ()
    ; other keywords
    ((_ condition b1 ...) ; _ is a shorthand for the keyword
      ; ... is a keyword for matching multiple statements
      (let loop ()
        ; expansion of p
        (when condition
          (begin
            b1 ...
            (loop))))))
; Usage
(while (condition)
  (intr1)
  (istr2))
```

Macros are expanded recursively, so we can have looping macros. Therefore the macro system is Turing complete.

```
(define-syntax my-let*
  (syntax-rules ()
    ((_ ((var val)) istr ...)
      ((lambda (var) istr ...) val))
    ((_ ((var val) . rest) istr ...)
      ((lambda (var)
        (my-let* rest istr ...)) val))))
```

Scheme macros are **hygienic**, this means that symbols in their definition are actually replaced with special unique symbols, meaning **it is impossible to have name clashes** once the marco is expanded.

Continuations

A continuation is an **abstract representation of the control state of the program**. The **current continuation** is the continuation that, from the perspective of running code, **would be derived from the current point in a program execution**.

Scheme natively supports continuations: **call-with-current-continuation (or call/cc) accepts a procedure with one argument, to which it passes the current continuation implemented as a closure**.

The **argument of the lambda** in call/cc is called an **escape procedure**. The escape can be **called with an argument that becomes the result of call/cc**. This means that the escape procedure abandons its own continuation and reinstates the continuation of call/cc.

```
(+ 3
  (call/cc
    (lambda (exit)
      (for-each (lambda (x)
```

```

        (when (negative? x)
          (exit x)))
      '(54 0 37 -3 245 19))
    10)))

```

An escape procedure has **unlimited extend**: if stored, it **can be called after the continuation has been invoked**.

We have two methods of implementing call/cc:

- **Garbage collected**: we handle function invocation via heap
 - We do not use the stack at all: **call frames are allocated on the heap**. Frames that are not used anymore are reclaimed by the GC. With this strategy call/cc simply saves the frame pointer of the current frame.
 - **Slows down all function calling**.
- **Stack strategy**: we use the stack as usual
 - When a call/cc is used, we create a **continuation object in the heap** by **copying the current stack**. When we call call/cc we need to **reinstate the saved stack, discarding the current one**.
 - It is **zero-overhead**: if we do not use call/cc we do not pay is cost
 - **The call/cc operation, however is very slow**.

Playing around: implementing exceptions

Most scheme dialects have their exception systems. We will try to roll our own.

```

(define *handlers* (list))
(define (push-handler proc)
  (set! *handlers* (cons proc *handlers*)))
(define (pop-handler)
  (let ((h (car *handlers*)))
    (set! *handlers* (cdr *handlers*))
    h))
(define (throw x)
  (if (pair? *handlers*)
      ((pop-handler) x)
      (apply error x)))

(define-syntax try
  (syntax-rules (catch)
    ((_ expl ... (catch what hand ...))
     (call/cc (lambda (exit)
                 (push-handler (lambda (x)
                                (if (equal? x what)
                                    (exit (begin hand ...))
                                    (throw x)))))
                 (let ((res (begin expl ...)))
                   (pop-handler)
                   res))))))

```

Object orientation

Quoting Alan Kay, the inventor of the term and Smalltalk:

OOP to me means only messaging, local retention and protection and hiding of state-process, and extreme late-binding of all things. It can be done in Smalltalk and in Lisp. There are possibly other systems in which this is possible, but I'm not aware of them.

Actually I made up the term “object-oriented”, and I can tell you I did not have C++ in mind.

The classical interpretation of OOP from C++ are based on the Simula programming language. Objective-C is a OO language that uses Smalltalk's concepts.

Closures

We can use closures to implement some basic OOP. We **define a procedure** which assumes the **role of a class**. This procedure, when called, **returns closure that works like an object**. **Access to the state is implemented through messages** to a function that works like a dispatcher.

```
(define (make-object)
  (let ((my-var 0))
    (define (my-add x)
      (set! my-var (+ my-var x))
      my-var)
    (define (get-my-var) my-var)
    (define (my-display)
      (newline)
      (display "my-var is: ")
      (display my-var)
      (newline))
    (lambda (message . args)
      (apply (case message
                ((my-add) my-add)
                (else (error "Unknown method!")))
              args))))
```

Inheritance can be achieved by **delegation**.

```
(define (make-son)
  (let ((parent (make-object))
        (name "an object"))
    (define (hello) "hi!")
    (define (my-display x)
      (display "My name is: ")
      (display name)
      (display " and")
      (parent 'my-display))
    (lambda (message . args)
      (case message
        ((hello) (apply hello args))
        ((my-display) (apply my-add args))
        (else (apply parent (cons message args)))))))
```

Prototypes

Take the concepts from Self: **there are no classes, new objects are obtained by cloning and modifying existing objects** (it inspired javascript). We will implement this model on top of scheme using **hash tables**.

```
(define new-object make-hash)
(define clone hash-copy)

; We could use functions, however we would need to pass `msg` already quoted
(define-syntax !! ;; setter
  (syntax-rules ()
    ((_ object msg new-eval)
     (hash-set! object 'msg new-eval))))
(define-syntax ?? ;; getter
  (syntax-rules ()
    ((_ object msg)
     (hash-ref object 'msg)))
(define-syntax -> ;; send message
  (syntax-rules ()
```

```

((_ object msg arg ...)
  ((hash-ref object 'msg) object arg ...))) ;; notice the first argument to
                                              ;; the method call

;;; Example
(define Pino (new-object))
(!! Pino name "Pino")
(!! Pino hello
  (lambda (self x) ;; defining a method. Notice the first argument
    (display (?? self name))
    (display (": hi, "))
    (display (?? x name))
    (display "!")
    (newline)))

```

Inheritance is not typical of systems like this. It can be achieved by **delegation**.

```

(define (deep-clone obj) ;; Recursively clone an object
  (if (not (hash-ref obj '<<parent>> #f))
      (clone obj)
      (let* ((cl (clone obj))
              (par (?? cl <<parent>>)))
        (!! cl <<parent>> (deep-clone (par))))))
(define (son-of parent) ;; creates a new object that is the son of a given
                        ;; parent
  (let ((o (new-object)))
    (!! o <<parent>> (deep-clone parent))
    o))

(define (dispatch object msg)
  (if (eq? object 'unknown)
      (error "Unknown message msg")
      (let ((slot (hash-ref object msg 'unknown)))
        (if (eq? slot 'unknown)
            (dispatch (hash-ref object '<<parent>> 'unknown) msg)
            slot)))))
(define-syntax ??
  (syntax-rules ()
    ((_ object msg)
     (dispatch object 'msg))))
(define-syntax ->
  (syntax-rules ()
    ((_object msg arg ...)
     ((dispatch object 'msg) object arg ...))))

```

Haskell

Mathematical functions do not have side-effects: the result depends entirely on the arguments. This is called **referential transparency**.

Scheme was mainly functional as some function have side-effects (set!). **Haskell**, instead, is **completely pure** and manages state and side-effects in a different way.

Evaluation

In a system without side-effects, the evaluation order of parameters does not matter.

Two evaluations of an expression differ in the order in which function applications are evaluated. A function application ready to be performed is called a reducible expression (**redex**).

Some strategies are:

1. **Innermost:** when there is more than one redex, **the leftmost one that does not contain other redexes is evaluated.**

With this strategy, **arguments of functions are always evaluated before evaluating the function itself.** This corresponds to **call-by-value**. It is the **faster** approach.

2. **Outermost:** dual to innermost, **we start with the redex that is not contained inside any other redex.**

With this strategy, **functions are always applied before their arguments.** This corresponds to **call-by-name**.

If there is an evaluation for an expression that terminates, call-by-name terminates and produces the same result (Church-Rosser confluence).

Call-by-need is a **memoized version of call-by-name** where, if the function argument is evaluated, that value is stored for subsequent uses. In a pure (side-effect-free) setting, this produces the same result as call-by-name and it is faster.

One way to **overcome non-termination** of possible evaluations is to **use thunks**. We can use a **force operation to evaluate it**. There is already an implementation of this strategy in racket based on `delay` and `force`. We will re-implement a version of it.

```
(struct promise
  (proc   ;; thunk
   value? ;; already evaluated?
   ) #:mutable)

(define-syntax delay
  (syntax-rules ()
    ((_ (expr ...))
     (promise (lambda () (expr ...)) #f)))) ; we created a thunk
(define (force promise)
  (cond
    ((not (promise? prom)) prom)
    ((promise-value? prom) (promise-proc prom))
    (else
     (set-promise-proc! prom ((promise-proc prom)))
     (set-promise-value?! prom #t)
     (promise-proc prom))))
```

In Haskell call-by-need is the default. If we need call-by-value, we need to force evaluation (we'll see how).

Currying

In Haskell **every function has one argument**. Functions with **multiple arguments are curried**. This means that we turn $f: N \times N \rightarrow N$ into $f: N \rightarrow (N \rightarrow N)$ (parenthesis can be omitted since $\rightarrow$ is right associative).

To illustrate how it works let us look at this scheme example:

```
(define (sum-square x)
  (lambda (y)
    (+ (* x x) (* y y))))
(define ((sum-square x) y)
  (+ (* x x) (* y y)))

(display ((sum-square 3) 5)) ;;; -> 34
```

This make partial application free.

Function definition

Functions are **declared through a sequences of equations**. This is also an example of **pattern matching**: arguments are matched with the right parts of equations, top to bottom. If a match succeeds, that definitions body is called.

```
length :: [Integer] -> Integer
length []      = 0
length (x:xs) = 1 + length xs
```

Type system

Haskell is **statically typed** with **type inference**: every well-typed expression is guaranteed to have a unique principal type and can be inferred automatically. It uses a **variant of the Hindley-Milner** type system (also used in other ML variants like F#).

In Haskell is possible to do **parametric polymorphism by declaring type variables**:

```
-- `a` is a type variable, `[a]` indicates a list of type `a` for any `a`
length :: [a] -> Integer
```

User-defined types

New types are introduced using data declarations.

```
-- This is like a tagged union type in C
data Bool = False | True
```

In the above, **Bool** is the type constructor, while **False** and **True** data constructors. These two constructors live in different namespaces so it is possible to use the same name for both:

```
-- This is like a struct in C
data Point a = Point a a
```

If we apply a **data constructor** we obtain a value (`Point 2.3 5.7`) while with a **type constructor** we obtain a type (`Point Float`).

Recursive types

Of course we can define recursive types. One example is the following:

```
data Tree a = Leaf a
            | Branch (Tree a) (Tree a)
-- data constructor Branch has type Branch :: Tree a -> Tree a -> Tree a

aTree :: Tree Char
aTree = Branch (Leaf 'a')
              (Branch (Leaf 'b') (Leaf 'c'))

-- example of a function working with Tree
fringe :: Tree a -> [a]
fringe (Leaf x)      = [x]
fringe (Branch l r) = fringe l ++ fringe r -- (++) is list concatenation
```

Lists are an example of recursive types. Using a scheme-like notation, they can be defined like:

```
data List a = Null
            | Cons a (List a)
```

Haskell has special syntax for lists: `[]` is both a data and type constructor, while `:` is an infix data constructor.

```
-- Pseudo-haskell
data [a] = []
        | a : [a]
```

Fields

Product types (like Point) are like structs in C or in Scheme. The access is positional via pattern-matching:

```
pointX Point x _ = x
pointY Point _ y = y
```

There is also a **C-like syntax to have named fields with auto-generated accessors:**

```
data Point = Point { pointX, pointY :: Float }
```

Synonyms

Synonyms are defined with type. They are usually used for shortness or readability.

```
type String = [Char]
type Assoc a b = [(a,b)]
```

Function composition and \$

(.) is mathematical function composition: $(f.g)(x) = f(g(x))$. \$ (apply operator) is an operator for avoiding some parenthesis, e.g $(10 *) (5 + 3) = (10 *) \$ 5 + 3$.

Infinite computations

Call-by-need is very convenient for dealing with infinite computations that provide data.

```
ones = 1 : ones -- infinite list of ones
```

```
numsFrom n = n : numsFrom (n + 1) -- equivalent to the [n..] notation
squares = map (^2) (numsFrom 0)
```

```
firstFiveSquares = take 5 squares -- perfectly legal to take a finite slice
```

List comprehensions are here. The syntax is inspired by that of the mathematical set theory:

```
l = [(x, y) | x <- [1,2], y <- "ciao"]
-- l = [(1, 'c'), (1, 'i'), (1, 'a'), (1, 'o'),
--      (2, 'c'), (2, 'i'), (2, 'a'), (2, 'o')]
```

```
fib = 1:1:[a + b | (a, b) -> zip fib (tail fib)]
```

Error

Bottom ($\perp$), is defined as bot. **All errors have value bot, a value shared by all types.** error :: String -> a is an outlier because it is polymorphic in the output. The reason is that it returns bot.

Pattern matching

We have already seen pattern matching: **it goes top-down, left-to-right.** Patterns **may have boolean guards.**

```
sign x | x > 0 = 1
      | x == 0 = 0
      | x < 0 = -1
```

Patterns have to be linear: each identifier has to be unique.

```
f (x:x:xs) = undefined -- IMPOSSIBLE
f (x:y:xs) | x == y = undefined -- POSSIBLE
```

Another way of doing pattern matching is by using case:

```
take m ys = case (m, ys) of
    (0, _)    -> []
    (_, [])   -> []
    (n, x:xs) -> x : take (n - 1) xs
```

```
-- equivalent to:
take 0 _      = []
take _ []     = []
take n (x:xs) = x : take (n - 1) xs
```

If-then-else

If-then-else is also present: if condition then then-clause else else-clause. N.B: with call-by-need we can define if as a function:

```
if True  x _ = x
if False _ y = y
```

let and where

let is similar to Scheme's letrec*. where is similar but reversed:

```
let x = 3
    y = 12
in x + y
```

```
powerset :: [a] -> [[a]]
powerset set = powerset' set [[]] where
    powerset' [] out = out
    powerset' (e:set) out = powerset' set (out ++ [e:x | x <- out])
```

Call-by-need and strictness

In scheme, we saw that **fold-left** is very **efficient** as it is **naturally tail recursive**. In Haskell this does not hold true: **due to call by need it is very memory intensive**.

```
foldl f z []      = z
foldl f z (x:xs) = foldl f (f z x) xs
```

Most of Haskell code uses foldr and this is one of the reasons.

There are **various ways to enforce strictness** in Haskell:

1. In **data types**: data Complex = Complex !Float !Float. It tells the compiler to **never store unevaluated expression inside those fields**. There are **extensions for using ! also for function parameters**.
2. We can force evaluation of a function using seq:: a -> t -> t. **The semantics of seq are the following: seq x y returns y only if x terminates.**

We can define a strict version of foldl:

```
foldl' f z []      = z
foldl' f z (x:xs) = let z' = f z x
                    in seq z' (fold' f z' xs)
```

There is a convenient strict variant of \$ called \$!.

Modules

Simple module, with import, export and namespaces.

```

module Test where -- export everything
...

module Tree (Tree(Leaf, Branch), fringe) where
...

module Main (main) where
import Tree (Tree(Leaf,Branch))
main = print (Branch (Leaf 'a') (Leaf 'b'))

```

Type classes

Type classes are the mechanism for providing **ad hoc polymorphism** (aka overloading).

Let us consider a function like `elem`. What should be its type?

```

x `elem` []      = False
x `elem` (y:ys) = x == y || (x `elem` ys)

```

We need to express the fact that `==` is defined for all types acceptable by the function. We can use the type class `Eq` and declare a type to be an instance of said class. If a type is instance of a class it means it implements all functions of said class.

```

class Eq a where
    (==) :: a -> a -> Bool
...

elem :: (Eq a) => a -> [a] -> Bool
...

```

`(Eq a)` is called a **constraint** on type `a`.

To define an instance for a class we use the following syntax:

```

-- we added a constraint on the instance: we can check equality of `Tree a` only
-- if `a` implements `Eq`
instance (Eq a) => Eq (Tree a) where
    Leaf a == Leaf b = a == b
    (Branch l1 r1) == (Branch l2 r2) = (l1 == l2) && (r1 == r2)
    _ == _ = False

```

The **implementation** of functions in classes are called **methods**.

There is **inheritance (multiple)** between classes (e.g. `Ord`):

```

class (Eq a) => Ord a where
...

```

Some classes can be **auto-implemented** by the compiler using the deriving keyword:

```

data Tree a = Leaf a | Tree a a
    deriving (Show,Eq)

```

IO

IO cannot be referentially transparent since it is based on state change. This means that if we perform a sequence of operations, they must be performed in order.

We introduce the **IO action**. IO is an **instance of the monad class**.

```

getChar :: IO Char
putChar :: Char -> IO ()

```

`main` is the default entry point of a program. For **working with actions we have some special syntax at our disposal**:

- `do` starts a block of ordered operations
- `<-` is used to obtain a value from an action

```
main :: IO ()
main = do
    putStr "Say something: "
    thing <- getLine
    putStrLn $ "You said \"\" ++ thing ++ "\"."
```

Example of program that reads a file and prints its contents:

```
import System.IO
import System.Environment

main = do
    args <- getArgs
    handle <- openFile (head args) ReadMode
    contents <- hGetContents handle -- note: hGetContents lazily reads the file
    putStr contents
    hClose handle
```

Exceptions

Haskell code can raise exceptions but to catch them we need an IO action. There are **different ways** of handling exceptions, the simplest one is `handle`

```
handle :: Exception e => (e -> IO a) -> IO a -> IO a

-- usage
main = handle handler readFile
    where handler e
        | isDoesNotExistError e = putStrLn "File does not exists"
        | otherwise = putStrLn "Something has gone wrong"
```

Maps and arrays

Some useful data structures are **arrays and hash-tables**. However, the way of interacting with said structures is imperative (there are libraries for using this style). The standard Haskell way is to use **immutable versions of this structures**: updates to them copy the structure, not change it.

```
import Data.Map
import Data.Array

exmap = let m = fromList [("nose", 1), ("emerald", 27)]
        n = insert "rug" 98 m
        o = insert "nose" 9 n
        in (m ! "emerald", n ! "rug", o ! "nose")
-- exmap -> (27, 98, 9)

exarr = let m = listArray (1,3) ["alpha", "beta", "gamma"] -- (1,3) is the range of indexes
        n = m // [(2,"Beta")] -- `//` executes a list of updates
        o = n // [(1,"Alpha"), (3,"Gamma")]
        in (m ! 1, n ! 2, o ! 1)
-- exarr -> ("alpha", "Beta", "Alpha")
```

Towards monads

We saw that IO is a type construct instance of Monad. In recent versions of GHC the **Monad class needs the introduction of: Foldable, Functor and Applicative** (Functor $\subset$ Applicative $\subset$ Monad).

Foldable

Used for folding. It **requires only the definition of foldr** (foldl can be derived from foldr). The basic idea is to **apply a function f to all the elements in a container, starting from a value z**.

Maybe The Maybe type is **like the Optional java type**: it has two values Just v and Nothing. It is a **simple example of an instance of Foldable**.

```
instance Foldable Maybe where
  foldr _ z Nothing = z
  foldr f z (Just x) = f x z
```

Functor

Functor is the **class of all the types that offer a map operation**. The map operation of functors is called fmap :: (a -> b) -> f a -> f b (just like map). Defining a mapping for containers is quite natural.

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just a) = Just (f a)
```

Well defined functors should obey the following rules (not checked by compiler, but necessary to have it make sense):

1. fmap id = id
2. fmap (f . g) = fmap f \$ fmap g (homomorphism)

Applicative functors

Applicative functors are an **extension of functors**. The definition of the class is peculiar:

```
class (Functor f) => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b -- called apply
```

If f is a container, the ideas are not too complex:

1. **pure takes a value and returns an f containing it**
2. **<*> is like fmap**, but instead of taking a function, **it takes an f containing functions**, to apply it to a suitable container of the same kind

Maybe is an applicative functor, the instance definition is trivial. **Lists are also functors**. Lets see if they are **also applicative**.

```
concat :: Foldable t => t [a] -> [a]
concat l = foldr (++) [] l -- [[1,2], [3], [4,5]] => [1,2,3,4,5]
```

```
concatMap :: Foldable t => (a -> [b]) -> t a -> [b]
concatMap f l = concat $ map f l
```

```
instance Applicative [] where
  pure x = [x]
  fs <*> xs = concatMap (\f -> map f xs) fs
```

```
-- Example of apply:
-- [(+1), (*2)] <*> [1,2,3] => [2,3,4,2,4,6]
```

Monads

Monads are **an algebraic data structure used to represent computations**, we will often call these computations actions. Monads allow the programmer to **chain actions together to build an ordered sequence, in which each action is decorated with additional processing rules provided by the monad and permed automatically**.

Monads can also be used to make imperative programming easier in a pure functional language.

```
class Applicative m => Moand m where
  -- Most important operation: bind. Sequentially composes two actions, passing
  -- any value produced by the first as an argument to the second
  (>=>) :: m a -> (a -> m b) -> m b

  -- Called then. Sequentially composes two actions, discarding any value
  -- produced by the first
  (>>) :: m a -> m b -> m b
  m >> k = m >=> \_ -> k

  -- Injects a value into the monadic type
  return :: a -> m a
  return = pure

  -- Fail with a message
  fail :: String -> m a
  fail s = error s
```

Let us look at a simple monad instance:

```
instance Monad Maybe where
  (Just x) >=> k = k x
  Nothing >=> _ = Nothing
  fail _ = Nothing
```

Monadic laws

1. return is the **identity element**:

$$\begin{aligned} (\text{return } x) >=> f &\iff f \ x \\ m >=> \text{return} &\iff m \end{aligned}$$

2. **Bind is associative**:

$$(m >=> f) >=> g \iff m >=> (\lambda x \rightarrow (f \ x \ >=> \ g))$$

Note: **monads are analogous to monoids** with $\text{return} = 1$ and $>=> = '='$.

The do notation

The do notation is used as **syntactic sugar to hide >=> and >>**. The translation of do is dictated by these two rules:

1. $\text{do } e1 ; e2 \iff e1 \ >> \ e2$
2. $\text{do } p \leftarrow e1 ; e2 \iff e1 \ >=> \ \lambda p \rightarrow e2$

The list monad

The **list type is an instance of the monad class**: it involves **joining together a set of calculations for each value in the list**. bind is defined as concatMap. The **underlying idea** is to represent **non-deterministic computation** with a set of possible results.

Using the list monad, we can define **list comprehensions** as:

```
[(x,y) | x <- [1,2,3], y <- [1,2,3]]
--- equivalent to
do x <- [1,2,3]
  y <- [1,2,3]
  return (x,y)
```

The State monad

The State monad is a generic type that **manages state**.

```
data State st a = State (st -> (st, a))
```

The constructor **takes a function** because the State takes two parameters and **we need unary type constructors**. The function **takes the current state, performs the computation and returns a tuple containing the new state and the result**.

First we need to instance **Functor**:

```
-- We apply `f` to the value of `a`
instance Functor (State st) where
  fmap f (State g) = State (\s -> let (s', x) = g s
                                   in (s', f x))
```

Then instance **Applicative**:

```
instance Applicative (State st) where
  pure x = State (\t -> (t, x))

  (State f) <*> (State g) =
    State (\state -> let (s, f') = f state
                      (s', x) = g s
                      in (s', f' x))
```

The same approach can be used for the **monad** definition:

```
instance Monad (State state) where
  State f >=> g = State (\old ->
    let (new, value) = f old
    State f' = g value
    in f' new)
```

An important aspect of this monad is that **monadic code does not get evaluated to data, but to a function!** (Note that State is a function and bind is function composition). To **get a value out** of the State monad we can define:

```
runStateM :: State state a -> state -> (state, a)
runStateM (State f) st = f st
```

To actually use the state contained in our monad, we can define **some utilities to access it**:

```
getState = State (\state -> (state, state))
putState new = State (\_ -> (new, ()))
```

Erlang

It is a **concurrent-oriented** programming language where **distribution is almost transparent**. The core is **functional**, but **more pragmatic** than Haskell (not pure). It is **dynamic typed** with syntax similar to Prolog.

The standard library (OTP) is built for distributed fault-tolerant applications and continuously running applications with code swap.

It runs on a **VM**, BEAM. Like the JVM, we have more languages running in it, mainly Elixir.

Not really mainstream, but still used in some big industrial applications.

The basics

Variables start with an **upper case letter**. Variables can be **bound only once**. Once a variable has been bound its values cannot change.

Atoms are like symbols in scheme. **Any character code is allowed in an atom.** Singly quoted sequences of characters are atoms. Unquoted atoms must be lowercase.

Tuples are used to store a fixed number of items. There is also the **concept of record (a.k.a structs)**, but it is simply **syntax sugar for tuples**.

```
{x, y}.
```

Lists are like in Haskell. Main difference | is cons.

```
[1, 2, 3].  
A ++ [4, 5].  
[0 | A].
```

Standard strings are a PITA. It is common to use libraries like bitstrings or UTF.

Comprehensions are more or less like Haskell.

```
{X, Y} || X <- [-1, 0, 1], Y <- [one, two, three], X >= 0].
```

Pattern matching is more powerful than Haskell's (**no linearity**), based on Prolog's.

```
A = 10. % succeeds - binds A to 10  
{A, A, B} = {abc, abc, foo}. % succeeds - binds A to abc and B to foo  
{A, A, B} = {abd, def, 123}. % fails  
[A, B | C] = [1, 2, 3, 4, 5, 6, 7]. % succeeds - binds A = 1, B = 2, C = [3,4,5,6,7]  
[H | T] = [abc]. % Succeeds - binds H = abc, T = []
```

Maps are first class. They are functional hash maps.

```
Map = #{one => 1, "Two" => 2, 3 => three}.  
Map#{one := "I"}. % update/insert  
#{ "Two" := V } = Map. % retrieves a value and puts it into V
```

Function calls are as expected: module:func(args...). If the function is inside this module we can avoid the module:.

Modules system is basic. Functions need to be exported to be available to the outside.

```
-module(demo). % '-' denotes a preprocessor directive  
                % macro calls start with '?'  
-export([double/1]). % fname/n denotes a function called 'fname' with 'n' arguments
```

```
double(X) -> times(X, 2).  
times(X, N) -> X * N.
```

Since it runs on a VM, we have shell erl, we can compile to bytecode with erlc while we run scripts with erlscript

Built-in functions are in the erlang module.

Function syntax

A function is **defined as a series of clauses**. Clauses are **scanned from top to bottom**. When a match is found, all variables in the head are bound. **Variables are local to each clause**.

The body is separated by ,. Clauses are separated by ;. Definition is ended with ..

```
-module(mathStuff)  
-export([factorial/1, area/1])  
  
factorial(0) -> 1;  
factorial(N) -> N * factorial(N-1),  
  
area({square, Side}) -> Side * Side;  
area({circle, Radius}) -> 3.14 * Radius * Radius;  
area({triangle, A, B, C}) ->  
    S = (A + B + C)/2,
```

```
math:sqrt(S * (S - A) * (S - B) * (S - C));
area(Other) -> {invalid_object, Other}.
```

when introduces a pattern guards. There is a **guard sub-language, because guards must be evaluated in constant time.**

The apply(Mod, Func, Args) invokes Mod:Func with the given arguments. Mod and Func must be expressions that evaluate to atoms. The ?MODULE macro expands to the current module name.

Case and If

```
case lists:member(a, X) of
  true -> ... ;
  false -> ...
end,

if
  integer(X) -> ... ;
  tuple(X) -> ... ;
  true -> ...      % like an 'else'
end,
```

case executes pattern matching. **if** is an **if**. Main difference between the two is that **if** uses the guard language while **case** takes any predicate.

Lambdas and higher order functions

Syntax for lambdas is like this: Square = fun(X) -> X*X end. We can use it like this: Square(3).. Lambdas can be passed to higher order functions, to pass standard non-lambda functions we need to prefix their name with fun and explicitly indicate the arity.

```
Square = fun (X) -> X*X end.
lists:map(Square, [1,2,3]). % returns [1,4,9]
lists:foldr(fun my_function/2, 0, [1,2,3]).
```

The actor model

Everything is an actor: an independent unit of computation. **Actors are inherently concurrent and can only communicate through messages.** Actors can be **created dynamically**. There is no requirement on the order of received messages.

Processes are represented through different actors communicating only through messages. Each actor is a **lightweight process, handled by the VM** and not scheduled by the OS. The VM also handles the multiple cores and the distribution of actors in a network.

Concurrent programming

We have **three primitives**:

1. spawn: **create** a new process
2. send (shortened to !): **sends a message** to a process through its identifier. The operation is async.
3. receive ... end: **extract**, from the first to the last, **a message** from a process's mailbox queue with the provided set of patterns. It **blocks if the queue is empty or if no message matches**.

A process's mailbox is persistent through the lifetime of it.

To spawn a process we call Pid2 = spawn(Mod, Func, Args). Note that Pid2 is local to the process that called spawn. To send a message to another process we call Pid2 ! {self(), foo}: we sent {Pid1, foo} to Pid2. Pid2 can receive the message when receive {From, Msg} -> Actions end is called.

A quick example:

```

-module(echo).
-export([go/0, loop/0]).

go() ->
  Pid2 = spawn(echo, loop, []),
  Pid2 ! {self(), hello},
  receive
    {Pid2, Msg} -> io:format("P1 ~w~n", [Msg])
  end,
  Pid2 ! stop.

loop() ->
  receive
    {From, Msg} ->
      From ! {self(), Msg},
      loop(); % functions are tail recursive
  stop -> true
  end.

```

If we want to **receive messages in a specified order**, we can put the respective receive blocks in sequence:

```

receive
  foo -> ...
end,
receive
  bar -> ...
end.

```

We can **register processes with a specific alias** using `register(Alias, Pid)`. For example:

```

start() ->
  Pid = spawn(?MODULE, server, []),
  register(analyzer, Pid).

analyze(Seq) ->
  ...

```

Client-server

```

% server code
-module(myserver).
server(Data) -> % Data is the server state
  receive
    {From, {request, X}} ->
      {R, Data1} = fn(X, Data),
      From ! {myserver, {reply, R}},
      server(Data1)
  end.

% client-side interface
-export([request/1]).
request(Req) ->
  myserver ! {self(), {request, Req}},
  receive
    {myserver, {reply, Rep}} -> Rep
  end.

```

Timeouts

The receive blocks has an **optional after block that executes an action if no action in the receive block is performed until the specified timeout.**

One simple use is to abuse the behaviour of receive and create sleep and suspend:

```
sleep(T) ->
  receive
  after
    T -> true
  end.
suspend() ->
  receive
  after
    infinity -> true
  end.
```

Building application using Erlang

The OTP provides useful “Behaviours”, which are pluggable ready-to-use patterns like “Server”, “Supervisor” etc.. The applications are structured in a **tree structure with supervisors and multiple workers for each one.** The processes follow the “**let it crash**” principle: if there is a problem, do not try to recover, simply crash.

The supervisor+“let it crash” architecture works thanks to **process linking**: all workers are linked to their supervisor. Linking means that **if any children dies, the father and all siblings are also killed.** We can **intercept the kill message by settings the trap_exit process flag.**

An example of a simple application:

```
main(Count) ->
  register(the_master, self()),
  start_master(Count),
  unregister(the_master),
  io:format("That's all.~n").

start_master(Count) ->
  process_flag(trap_exit, true),
  create_children(Count),
  master_loop(Count).

create_children(0) -> ok;
create_children(N) ->
  Child = spawn_link(?MODULE, child, [0]),
  io:format("Child ~p created~n", [Child]),
  Child ! {add, 0},
  create_children(N-1).

master_loop(Count) ->
  receive
  {value, Child, V} ->
    io:format("child ~p has value ~p ~n", [Child, V]),
    Child ! {add, rand:uniform(10)},
    master_loop(Count);
  {'EXIT', Child, normal} ->
    io:format("child ~p has ended~n", [Child]),
    if
      Count == 1 -> ok;
      true -> master_loop(Count-1)
    end
  end
```

```

    end;
    {'EXIT', Child, _} ->
        NewChild = spawn_link(?MODULE, child, [0]),
        io:format("child ~p has died, now replaced by ~p ~n", [Child, NewChild]),
        NewChild ! {add, rand:uniform(10)},
        master_loop(Count)
end.

child(Data) ->
    receive
        {add, V} ->
            NewData = Data+V,
            BadChance = rand:uniform(10) < 2,
            if
                BadChance -> error("I'm dying"); % random error in child
                NewData > 30 -> ok; % child ends naturally
                true -> the_master ! {value, self(), NewData}, % there is still work to do
                    child(NewData)
            end
    end
end.

```

Rust

Memory safety

Characterizes programs everywhere and characterizes whether allocation/deallocation disrupts execution of the program. System level languages cannot afford garbage collection to ensure safety. Rust and C++ have compile-time and runtime mechanisms for ensuring such behaviour.

Memory safety bugs can be seen as an ownership problem: when a variable has a pointer to a region it own it.

Single ownership

A memory leak is a pointer to a region of memory that gets lost, making that region unfreeable.

C++ uses idiom RAII (Resource Allocation Is Initialization) to ensure the absence of memory leaks. It links the existence of object to the lifetime of automatic variables (through the use of smart pointers like `shared_ptr` and `unique_ptr`).

Ownership can be moved between variables throughout the lifetime of the program.

Multiple ownership

Multiple owners to the same region of memory can have overlapping lifetimes, but we need to ensure that the memory is accessed consistently. Two errors that can occur are:

1. Double free: we free the region two times
2. Use-after-free: we use a deallocated region after we freed it.

Rust's solution

Rust enforces the single-owner policy and has a strict policy about borrowing references (aliasing). RAII idiom is also used everywhere. We can also put arbitrary data using `Box<T>`.

Move semantics are the default and are implicit. Assigning a variable always moves ownership and we need to explicitly specify when we want to copy values. Move semantics are enforced also during function calls.

You can use `Arc<T>` (`shared_ptr`-like) to handle multiple ownership, but the more idiomatic way is borrowing.